



OCEAN UNIVERSITY OF CHINA

EXPERIMENT 3

Personal Lab Report

**Task Control, Robotic Arm Simulation
and Closed-Loop Sorting Integration**

Name: Cao Jingwen
Student ID: 24020036005
Group: 5
Responsibility: Part 2 – Task Control and Robotic Arm Simulation

September 2026

Robotics Experiment

I Basic Experimental Requirements

Experiment 3 requires the group to construct a complete desktop object-sorting system that connects visual recognition, task judgment, robotic grasping, classification, and exception handling. The complete group project contains both simulation and real-machine stages.

For the simulation stage, the main requirements are:

- Recognize at least two object categories.
- Use fixed pickup grids and determine the object/grid information from the vision output.
- Map the recognized object class to the correct destination bin.
- Control the robotic arm and gripper to complete grasping, transport, placement, and return-to-home motion.
- Repeat the sorting process automatically without manually selecting the target during task execution.
- Use ROS 2 interfaces to connect the vision, task-control, and robotic-arm modules.
- Provide exception handling and clear execution logs.
- Integrate the main simulation modules into one launch file.
- Complete a multi-object closed-loop sorting demonstration.

For Group 5, the two object classes used in the experiment are Square and Circle, with six fixed pickup grids and two destination bins.

II Individual Responsibilities

Experiment 3 was completed by four group members. The final division of work is shown in Figure 1.

- **Part 1 – Vision and Simulation Vision:** Square/Circle dataset, YOLO training, ROS 2 detector node, and simulated-camera recognition.
- **Part 2 – Task Control and Robotic Arm Simulation:** Grid judgment, bin assignment, state machine, exception handling, ROS 2 Action, mechanical-arm/gripper simulation, simulated grasping and classification, and complete simulation integration.
- **Part 3 – Real Vision and Tabletop Calibration:** real overhead RGB camera, six-grid calibration, real-image detection, and image-to-grid calibration.
- **Part 4 – Real Robotic Arm and System Integration:** real mechArm control, fixed pickup points, bin positions, gripper control, safety positions, exception stop, launch configuration, and final real-system integration.

My responsibility was Part 2. My work connected the Part 1 vision output with the simulated robotic-arm execution layer. I mainly completed:

- the six-grid/two-bin sorting scene and arm-control configuration;
- Grid judgment and class-to-bin assignment;
- task-manager state-machine design;
- the `/pick_and_place` ROS 2 Action interface;
- simulated arm/gripper grasping, transportation, placement, and return-to-home motion;
- exception handling and execution-state logging;
- integration of Part 1 visual detections into the task-control loop;

- image-to-grid calibration for the final camera view;
- one-launch integration and final six-object closed-loop verification.

你继续负责 **Part 2** 没问题，我们把原来 Part 3 里的**机械臂仿真部分并到你这里**。

新的四人分工

Part	主要阶段	负责内容
Part 1: 视觉 + 仿真视觉	 仿真	A: <code>Square/Circle</code> 数据集、YOLO训练、ROS 2检测节点、仿真相机识别
Part 2: 任务控制 + 机械臂仿真	 仿真	你: Grid判断、Bin分配、状态机、异常处理、ROS 2 Action、机械臂/夹爪仿真、仿真抓取分类、完整仿真联调
Part 3: 真机视觉 + 桌面标定	 真机	真实俯视RGB相机、真实6 Grid划分、YOLO真机识别、图像/Grid标定、真机视觉调试
Part 4: 真机机械臂 + 系统联调	 真机	mechArm真机、6个固定抓取点、Bin 1/2放置点、夹爪控制、安全位置、异常停止、Launch和最终真机联调

Figure 1: Final division of work for Experiment 3

III Project Implementation Principle

3.1 Overall System Architecture

The Part 2 simulation system is organized as a modular ROS 2 pipeline:

Simulated Camera → YOLO Detector → `/detections` → Task Manager → `/pick_and_place` → Action Server → mechArm270 + Gripper

The vision module provides class IDs, confidence scores, and bounding-box centers. The task manager uses these values to identify the Grid, determine the destination Bin, and send a pick-and-place goal to the robotic-arm Action Server. The Action Server then executes the corresponding grasping and placement motion.

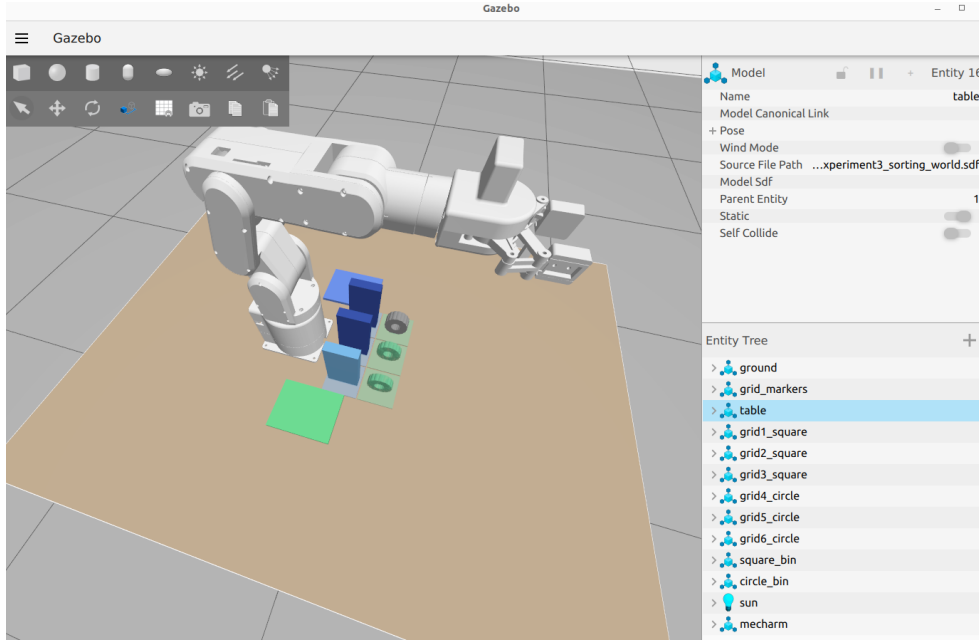


Figure 2: Complete Gazebo sorting scene with six targets and two destination bins

3.2 Vision Interface and Class Mapping

The vision module publishes:

Topic: /detections
Type: vision_msgs/msg/Detection2DArray

The bounding-box center is read from:

detection.bbox.center.position.x
detection.bbox.center.position.y

The class and confidence are read from:

result.hypothesis.class_id
result.hypothesis.score

The original YOLO training model internally uses raw class IDs in a different order, but the ROS 2 detector node publishes the unified Group 5 convention:

class_id = 0 → Square
class_id = 1 → Circle

The task manager therefore directly follows the published ROS 2 class convention.

Table 1: Class-to-bin mapping used in Part 2

Published Class ID	Class Name	Final Test Grids	Destination
0	Square	G1–G3	Bin 1
1	Circle	G4–G6	Bin 2

3.3 ROS 2 PickAndPlace Action

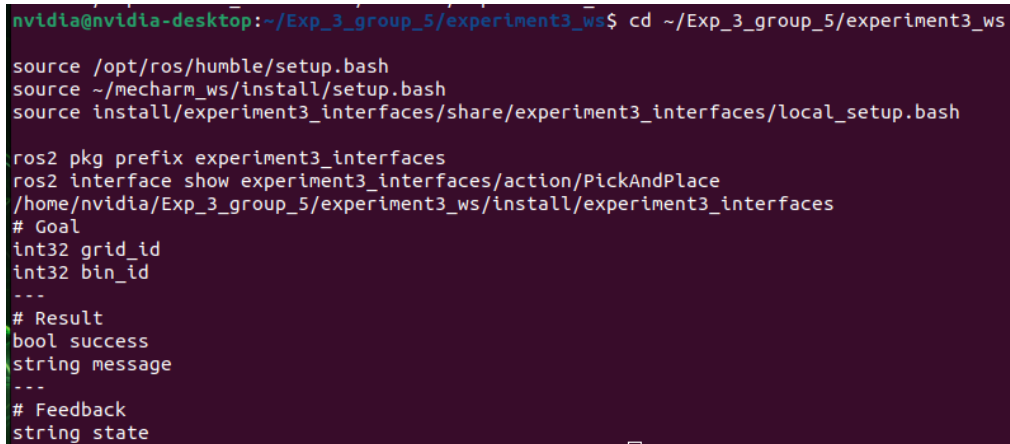
The task manager communicates with the arm-control module through a custom ROS 2 Action:

```
Action topic:
/pick_and_place

Action type:
experiment3_interfaces/action/PickAndPlace

Goal:
int32 grid_id
int32 bin_id
---
Result:
bool success
string message
---
Feedback:
string state
```

This design separates the high-level task logic from the low-level arm execution. It also provides a clear interface for later real-robot integration.

A terminal window with a dark purple background and light green text. The user is at a prompt 'nvidia@nvidia-desktop:~/Exp_3_group_5/experiment3_ws\$' and has entered several commands to set up and verify the ROS 2 environment. The output shows the package name 'experiment3_interfaces' and the action interface 'PickAndPlace' with its goal, result, and feedback fields.

```
nvidia@nvidia-desktop:~/Exp_3_group_5/experiment3_ws$ cd ~/Exp_3_group_5/experiment3_ws
source /opt/ros/humble/setup.bash
source ~/mecharm_ws/install/setup.bash
source install/experiment3_interfaces/share/experiment3_interfaces/local_setup.bash

ros2 pkg prefix experiment3_interfaces
ros2 interface show experiment3_interfaces/action/PickAndPlace
/home/nvidia/Exp_3_group_5/experiment3_ws/install/experiment3_interfaces
# Goal
int32 grid_id
int32 bin_id
---
# Result
bool success
string message
---
# Feedback
string state
```

Figure 3: Verification of the PickAndPlace ROS 2 Action interface

3.4 Grid and Bin Reachability

Before the full sorting loop was executed, I verified that the six pickup grids and both destination bins were reachable by the simulated mechArm270. The final IK test confirmed that the required above-target and target poses were reachable for all pickup and placement locations.

```

===== EXPERIMENT 3 FINAL LAYOUT IK TEST =====

G1  ABOVE OK   radius=0.120 m   q=[ 0.543  0.143 -0.43  0.002  1.861  0.544]
G1  AT   OK   radius=0.149 m   q=[0.543  0.126  0.21  0.005  1.231  0.541]
G2  ABOVE OK   radius=0.104 m   q=[-0.   -0.029 -0.236 -0.   1.836 -0.   ]
G2  AT   OK   radius=0.137 m   q=[-5.000e-03 -4.000e-02  3.730e-01  3.000e-03  1.238e+00  3.136e+00]
G3  ABOVE OK   radius=0.120 m   q=[-0.536  0.139 -0.421 -0.004  1.859  1.034]
G3  AT   OK   radius=0.149 m   q=[-0.538  0.126  0.21  0.005  1.231 -0.54 ]
G4  ABOVE OK   radius=0.179 m   q=[ 0.359  0.73  -0.991 -0.   1.831  0.359]
G4  AT   OK   radius=0.208 m   q=[ 0.359  0.736 -0.397  0.   1.232  0.359]
G5  ABOVE OK   radius=0.169 m   q=[ 1.000e-03  5.620e-01 -7.280e-01 -1.000e-03  1.736e+00  0.000e+00]
G5  AT   OK   radius=0.199 m   q=[-0.   0.626 -0.219  0.   1.164  0.   ]
G6  ABOVE OK   radius=0.179 m   q=[-0.359  0.73  -0.991 -0.   1.831 -0.359]
G6  AT   OK   radius=0.208 m   q=[-0.359  0.736 -0.397  0.   1.232 -0.359]
BIN1 ABOVE OK   radius=0.153 m   q=[ 1.141  0.36  -0.445 -0.002  1.655 -0.43 ]
BIN1 AT   OK   radius=0.186 m   q=[ 1.138e+00  4.640e-01  2.900e-02  1.000e-03  1.077e+00 -4.330e-01]
BIN2 ABOVE OK   radius=0.153 m   q=[-1.139  0.367 -0.456 -0.   1.659  0.432]
BIN2 AT   OK   radius=0.186 m   q=[-1.139e+00  4.640e-01  2.900e-02  1.000e-03  1.077e+00  4.320e-01]

=====
FINAL RESULT: ALL TARGETS REACHABLE
=====

```

Figure 4: IK reachability verification for all Grids and Bins

3.5 Mechanical-Arm Motion Sequence

Each sorting cycle follows the same general motion sequence:

HOME → above grid → descend → close gripper → lift → move to bin → release →
retreat → HOME

The final general arm move time was adjusted to 4.0 s and the gripper move time remained 1.2 s. Critical approach and return motions were kept conservative to maintain stable grasping.

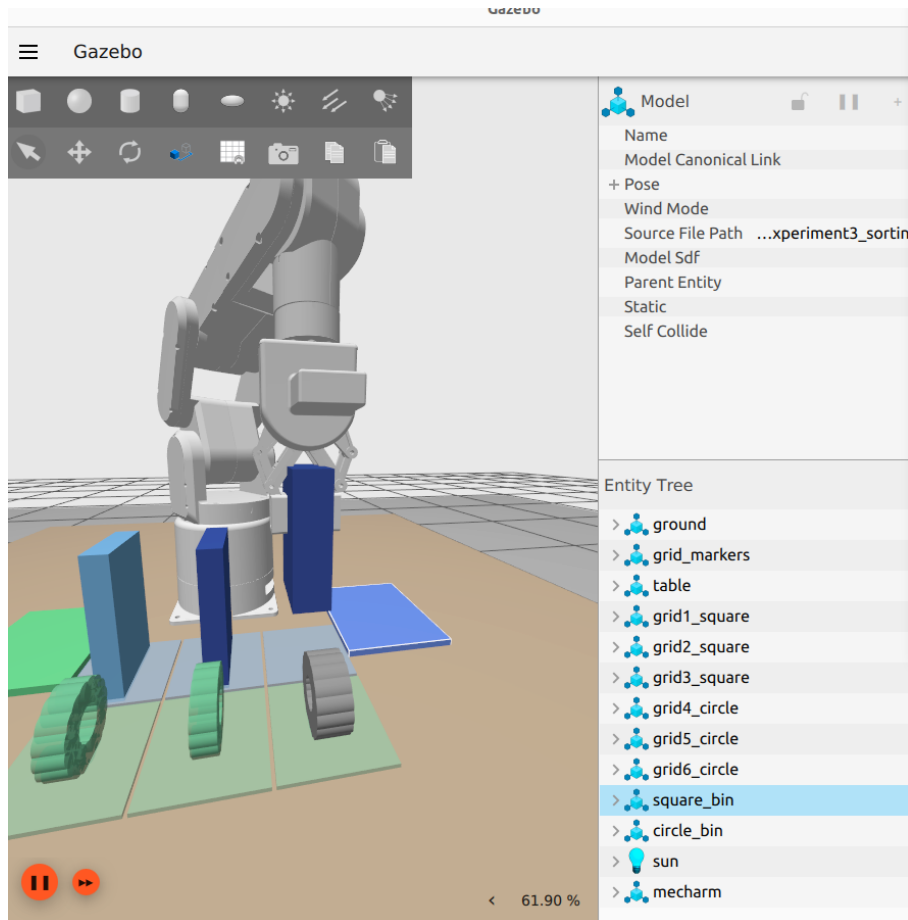


Figure 5: Successful simulated grasp and lift of a Square object

3.6 Task-Manager State Machine

The main task states are:

- WAITING
- TARGET_SELECTED
- REQUEST_PICK
- WAIT_RESULT
- SUCCESS
- GRASP_FAILED
- DONE

The normal state flow is:

WAITING $\rightarrow$ TARGET_SELECTED $\rightarrow$ REQUEST_PICK $\rightarrow$ WAIT_RESULT $\rightarrow$ SUCCESS $\rightarrow$ WAITING

If an Action fails, the corresponding Grid is not marked as completed, so the control flow can retry it when a valid target is available again.

3.7 Image-to-Grid Calibration

The first implementation used a simple 3×2 image division. After the final camera was adjusted to an angled view, the six targets no longer occupied six equally spaced image regions. Therefore, the image-to-grid relation was calibrated using the actual target centers in the final view.

The approximate calibrated centers were:

Table 2: Final image-space Grid calibration

Grid	u (pixel)	v (pixel)
G1	371.2	218.1
G2	317.7	207.8
G3	268.0	194.5
G4	347.8	256.6
G5	290.9	242.3
G6	242.6	227.5

For a detection center (x, y) , the task manager computes the squared distance to every calibrated Grid center:

$$d_i^2 = (x - g_{x,i})^2 + (y - g_{y,i})^2$$

and associates the target with the nearest valid Grid. This makes the Grid judgment consistent with the final camera viewpoint.

3.8 Sequential Closed-Loop Scheduling

To make the sorting procedure deterministic and easy to verify, I designed the task manager to process the expected Grids in the sequence:

G1 $\rightarrow$ G2 $\rightarrow$ G3 $\rightarrow$ G4 $\rightarrow$ G5 $\rightarrow$ G6

The visual detector still provides the object class and target information. The sequential scheduler only determines which Grid should be processed next, preventing overlapping or repeated task requests.

In the final test scene, G1–G3 were Square samples sent to Bin 1, and G4–G6 were Circle samples sent to Bin 2.

3.9 One-Launch Integration

The final simulation launch file integrates the major modules of Part 2, including the Gazebo world, camera bridge, YOLO detector, PickAndPlace Action Server, task manager, robot state publisher, robot spawning, arm controller, and gripper controller.

The final simulation is launched with:

```
ros2 launch experiment3_arm_sim experiment3_sim.launch.py
```

IV Problems Encountered and Solutions

4.1 Problem 1: Correct Arm Motion Did Not Always Produce a Stable Grasp

At the beginning of the simulation tests, the arm could reach the target and the gripper could close, but the object did not always move together with the gripper. In several trials the target slipped, rotated, or stayed on the table.

I did not immediately rewrite the complete motion program. Instead, I checked the problem layer by layer: target position, end-effector height, gripper opening/closing range, collision size, contact/friction settings, and the lifting trajectory after grasping. I also adjusted the Circle-object collision parameters while keeping its visual appearance suitable for recognition.

Personal thinking: this problem made me realize that “the trajectory is correct” and “the manipulation task succeeds” are different conditions. Stable grasping depends on kinematics, geometry, collision, and contact physics together.

4.2 Problem 2: Circle Recognition Confidence Was Unstable in the Simulation Scene

During the integration stage, Square objects were detected reliably, while Circle objects initially had much lower confidence in the Gazebo scene. The main reason was that the simulated object appearance differed from the image distribution used by the original detector.

I first verified that the detector itself worked normally, then adjusted the simulation visual appearance and coordinated the detector operating parameters until both classes could be recognized stably in the final scene. After the final integration, the six targets were detected with high confidence, and the working threshold was set to 0.80 to suppress unnecessary low-confidence boxes during arm motion.

Personal thinking: when a vision result becomes unstable after moving from one scene to another, it is important to first determine whether the problem comes from the model, the data distribution, or the ROS 2 communication layer instead of changing all modules at the same time.

4.3 Problem 3: The Original 3×2 Image Division Became Invalid After Camera Adjustment

The original Grid logic assumed that the 640×480 image could be divided evenly into six regions. After the camera was changed to a more useful angled view, this assumption no longer matched the real image geometry. The detector could see the objects correctly, but several targets were mapped to the wrong Grid.

I solved this by recording the actual target centers in the final image and replacing the equal image partition with calibrated nearest-center Grid mapping.

Personal thinking: this problem helped me distinguish object recognition from spatial interpretation. A

detector can be completely correct while the geometric mapping used by the control system is still wrong.

4.4 Problem 4: Task Execution Needed a Stable and Reproducible Order

When several valid targets exist in one frame, the control layer must avoid sending overlapping or rapidly changing task requests. I therefore used a deterministic Grid sequence G1–G6. The task manager waits for the expected Grid, verifies its detection result, sends one Action request, waits for the result, and only then continues to the next Grid.

Personal thinking: for an engineering demonstration, a clear and reproducible scheduling rule is often more valuable than unnecessary complexity. The important point is that the task remains automatically driven by the vision result and the ROS 2 state machine.

4.5 Problem 5: Motion Speed Had to Be Improved Without Damaging Stability

After the full sorting process became stable, I wanted the final demonstration to run slightly faster. However, directly shortening every trajectory duration could cause unstable grasping or abrupt motion.

I therefore changed the general arm move time gradually from 4.5 s to 4.0 s while keeping critical approach and return motions conservative. The complete six-object process remained stable after this adjustment.

Personal thinking: once a stable baseline has been obtained, parameter optimization should be incremental. Changing one factor at a time makes the effect easier to evaluate and reduces the risk of losing a previously successful configuration.

V Experimental Results

The final Part 2 system successfully completed automatic Square and Circle sorting using the integrated vision–task–Action–robot pipeline.

A successful Square-object closed-loop cycle is shown in Figure 6, and a successful Circle-object cycle is shown in Figure 7.

```
[INFO] [1789704498.316057358] [experiment3_task_manager]: ACTION_FEEDBACK | RETURNING_HOME
[INFO] [1789704506.220836588] [experiment3_task_manager]: STATE: WAIT_RESULT -> SUCCESS
[INFO] [1789704506.221795683] [experiment3_task_manager]: SUCCESS | Grid 1 -> Bin 1 | processed=[1]
[INFO] [1789704506.223616462] [experiment3_task_manager]: STATE: SUCCESS -> WAITING
```

Figure 6: Task-manager closed-loop success for a Square object

```
[INFO] [1789704694.090604713] [experiment3_task_manager]: ACTION_FEEDBACK | RETURNING_HOME
[INFO] [1789704701.186328506] [experiment3_task_manager]: STATE: WAIT_RESULT -> SUCCESS
[INFO] [1789704701.187170023] [experiment3_task_manager]: SUCCESS | Grid 6 -> Bin 2 | processed=[1, 6]
[INFO] [1789704701.187698031] [experiment3_task_manager]: STATE: SUCCESS -> WAITING
```

Figure 7: Task-manager closed-loop success for a Circle object

The final sorted simulation scene is shown in Figure 8.

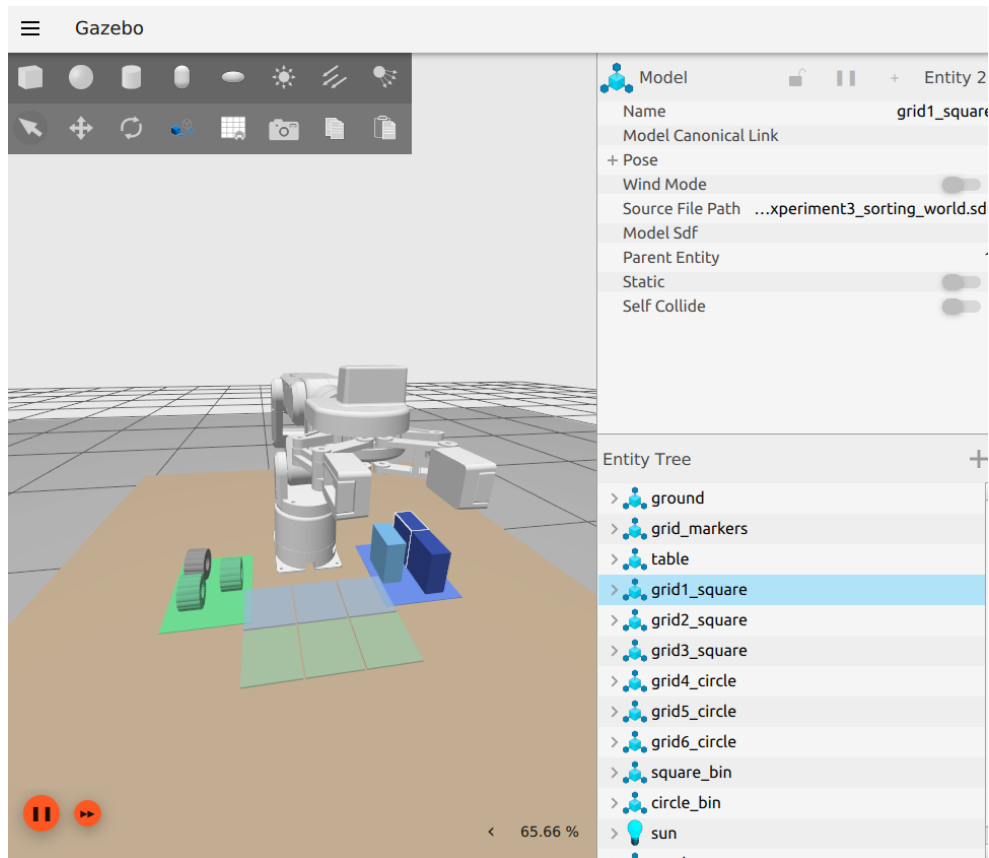


Figure 8: Final simulation scene after object sorting

The complete closed-loop test ended with all six Grids processed successfully:

```
processed=[1, 2, 3, 4, 5, 6]
DONE: 6/6 grids processed.
```

```
[pick_and_place_server-5] [INFO] [1789722503.043052693] [lws_transfer_node]: SUCCESS: Grid 6 -> Bin 2 pick-and-place completed
[task_manager-6] [INFO] [1789722503.046674142] [experiment3_task_manager]: STATE: WAIT_RESULT -> SUCCESS
[task_manager-6] [INFO] [1789722503.047433716] [experiment3_task_manager]: SUCCESS | Grid 6 -> Bin 2 | processed=[1, 2, 3, 4, 5, 6]
[task_manager-6] [INFO] [1789722503.048308270] [experiment3_task_manager]: STATE: SUCCESS -> DONE
[task_manager-6] [INFO] [1789722503.049773624] [experiment3_task_manager]: =====
[task_manager-6] [INFO] [1789722503.050795542] [experiment3_task_manager]: DONE: 6/6 grids processed.
[task_manager-6] [INFO] [1789722503.051745649] [experiment3_task_manager]: =====
```

Figure 9: Final six-object vision-based closed-loop sorting result

The final result verifies:

- stable Square and Circle recognition in the final simulation scene;
- correct Grid judgment and class-to-bin assignment;
- ROS 2 Action request, feedback, and result handling;
- stable grasping, transport, release, and return-to-home motion;
- sequential G1–G6 task execution;
- completion of all six objects with the final DONE state.

VI Team Collaboration and Handover

My Part 2 work was located between the Part 1 perception module and the later real-system stages.

The main interface from Part 1 to Part 2 is:

`/detections` with `0 = Square, 1 = Circle`

The main interface from Part 2 to the robotic-arm execution layer is:

`/pick_and_place` with `grid_id, bin_id`

Keeping these interfaces stable allows Part 3 to replace the simulated camera with a real camera and Part 4 to replace the simulated arm execution with a real mechArm implementation without redesigning the whole task-control logic.

VII Personal Reflection

This experiment gave me a deeper understanding of complete-system integration than a single-module programming task.

The most important lesson was that the final robotic behavior depends on the consistency of multiple layers. Correct visual recognition does not automatically guarantee correct Grid judgment. Correct Grid judgment does not automatically guarantee successful grasping. Correct arm motion does not automatically guarantee stable physical interaction.

During debugging, I gradually formed a more systematic method:

`perception` → `mapping` → `decision` → `communication` → `motion` → `grasping`

I learned to verify each layer separately before changing the next layer. This reduced unnecessary repeated modification and made the source of each problem easier to identify.

I also gained a clearer understanding of the importance of stable ROS 2 interfaces in group collaboration. By keeping the class convention, `/detections`, Grid IDs, and `/pick_and_place` consistent, the later real-camera and real-robot stages could continue from the completed Part 2 system instead of rebuilding the whole pipeline.

VIII Conclusion

In Experiment 3, I was mainly responsible for Part 2: task control and robotic-arm simulation.

I completed Grid judgment, Square/Circle bin assignment, state-machine design, exception handling, ROS 2 Action communication, mechArm270/gripper simulation, image-to-grid calibration, one-launch integration, and the final closed-loop sorting process.

The final simulation connected:

`YOLO Detection` → `Grid/Category Decision` → `PickAndPlace Action` → `Robotic Sorting`

and successfully processed G1 through G6, reaching the final DONE state with all six objects completed.

This Part 2 work completed the simulation task-control stage and provided a stable interface foundation for the later real-camera calibration and real-mechArm integration stages.